

Graham Pollak Theorem

Imagine you have a tree, i.e. a connected graph with no cycles. For this tree, we can create a matrix where each entry shows the number of edges you need to travel to get from one vertex (point) to another. For example, in a simple 3-vertex tree, the distance between connected points is 1, and between the two end points is 2. Suppose it has n vertices. For such a tree, you can form a $n \times n$ distance matrix. The determinant of this matrix turns out to depend only on the number of vertices, not on the specific shape of the tree. And that fact is essentially what the Graham–Pollak theorem describes!

The Graham-Pollak theorem reveals a fascinating property about trees (a type of graph without any cycles) and their distance matrices. The Graham–Pollak theorem—originally from 1971—can be phrased like this:

If you label the vertices of a tree with numbers $1, 2, \dots, n$, and form the matrix $D = [d(i, j)]$ where each entry $d(i, j)$ is the distance (number of edges) between vertex i and vertex j , then the determinant of D depends only on n , not on how the tree looks. In fact, the determinant is:

$$\det(D) = (-1)^{(n-1)}(n-1)2^{(n-2)}$$

So for any tree with n vertices—whether it's a path, a star, or something spiky—when you fill in the distance matrix, perform reduction operations and calculate the determinant, you'll always get that same number.

Let us try to see this with an example of a tree with 8 vertices

(Take note that the indexing in matrix starts from 0 . Hence all the steps are performed accordingly.)

First let us define function to get the determinant of resulting distance matrix by Schur Complement

```
In [87]: def schur_determinant(M):

    n = M.nrows()

    # Standard approach: A is (n-1)×(n-1), B is (n-1)×1, C is 1×(n-1), D is 1×1
    A = M[1:, 1:]
    print (f"let A be:\n{A}") # (n-1)×(n-1)
    C = M[1:, 0]
    print (f"let C be:\n{C}")      # (n-1)×1
    B = M[0, 1:]      # 1×(n-1)
    print (f"let B be:\n{B}")
    D = M[0, 0]      # 1×1 (scalar)
    print (f"let D be:\n{D}")

    # Schur complement: S = D - C*A^(-1)*B
    schur_comp = D - B * A.inverse() * C
    L = det(A) * det(schur_comp)
    print(f"schur_comp = D - B*A.inverse()* C")# This is a scalar
    print(f"D2= det(A) * det(schur_comp)")
    return round(L)
```

```
In [86]: #number of vertices
n = 8
# Define the edges and graph
edges = [(1,2),(2,4),(3,4),(4,5),(5,7),(6,7),(7,8)]
T = Graph(edges)
print("The graph is given as:")
T.show()
# Compute the distance matrix
D = T.distance_matrix()
print("The Distance matrix is:")
print(D)

for v in T.vertices():
```

```

vertices_with_degree_1 = [v for v in T.vertices() if T.degree(v) == 1]

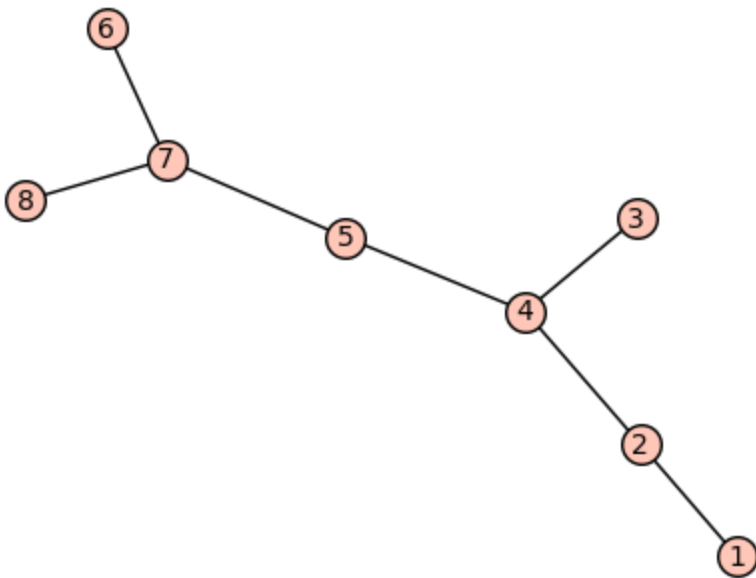
if vertices_with_degree_1: # Check if the list is not empty
    vertex_to_delete = vertices_with_degree_1[-1]
    print(f" D[0,{vertex_to_delete-1}] < D[0,
{T.neighbors(vertex_to_delete)[0]-1}]"
        if D[0, vertex_to_delete-1] > D[0, T.neighbors(vertex_to_delete)
[0]-1]:
        print(f"Consider vertex pendant v{vertex_to_delete}. \nPerform the
following operations:")
        print(f"C{vertex_to_delete} -> C{vertex_to_delete} -
C{T.neighbors(vertex_to_delete)[0]}")
        print(f"R{vertex_to_delete} -> R{vertex_to_delete} -
R{T.neighbors(vertex_to_delete)[0]}")
        D[vertex_to_delete-1, :] = D.row(vertex_to_delete-1) -
D.row(T.neighbors(vertex_to_delete)[0]-1)
        D[:, vertex_to_delete-1] = D.column(vertex_to_delete-1) -
D.column(T.neighbors(vertex_to_delete)[0]-1)
        print(D)
        print(f"Now remove the vertex v{vertex_to_delete} from the graph.
The tree will look like:")
        T.delete_vertex(vertex_to_delete)
        T.show()

    else:
        print("No vertices with degree 1 found.") # Handle case with no
degree 1 vertices

print(f"Finally we have the Determinant of D2 (by Schur complement) :
{schur_determinant(D)}")
c = (-1)**(n-1) * (n-1) * 2**(n-2)
print("The determinant from the formula: ", c)

```

Out[86]: The graph is given as:



The Distance matrix is:

```

[0 1 3 2 3 5 4 5]
[1 0 2 1 2 4 3 4]
[3 2 0 1 2 4 3 4]
[2 1 1 0 1 3 2 3]
[3 2 2 1 0 2 1 2]
[5 4 4 3 2 0 1 2]
[4 3 3 2 1 1 0 1]
[5 4 4 3 2 2 1 0]

```

D[0,7] < D[0,6]

Consider vertex pendant v8.

Perform the following operations:

C8 -> C8 - C7

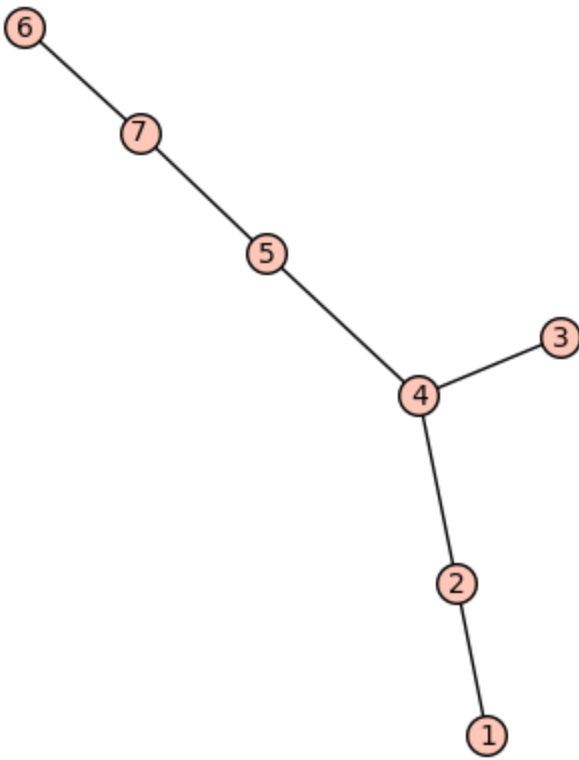
R8 -> R8 - R7

```

[ 0  1  3  2  3  5  4  1]
[ 1  0  2  1  2  4  3  1]
[ 3  2  0  1  2  4  3  1]
[ 2  1  1  0  1  3  2  1]
[ 3  2  2  1  0  2  1  1]
[ 5  4  4  3  2  0  1  1]
[ 4  3  3  2  1  1  0  1]
[ 1  1  1  1  1  1  1 -2]

```

Now remove the vertex v8 from the graph. The tree will look like:



$$D[0,5] < D[0,6]$$

Consider vertex pendant v6.

Perform the following operations:

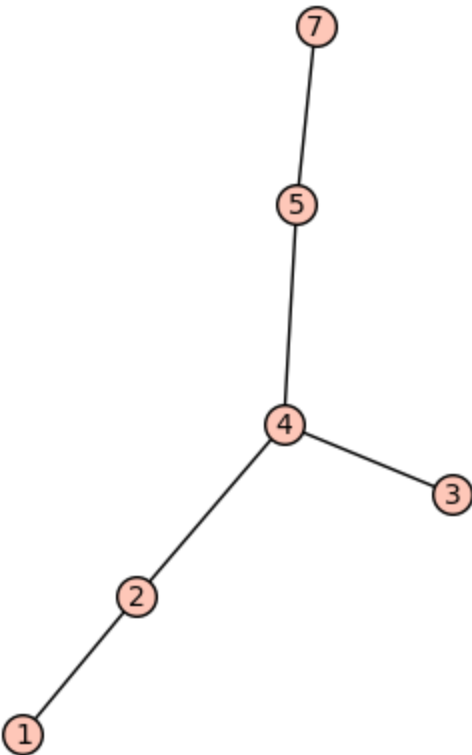
$$C6 \rightarrow C6 - C7$$

$$R6 \rightarrow R6 - R7$$

```

[ 0  1  3  2  3  1  4  1]
[ 1  0  2  1  2  1  3  1]
[ 3  2  0  1  2  1  3  1]
[ 2  1  1  0  1  1  2  1]
[ 3  2  2  1  0  1  1  1]
[ 1  1  1  1  1 -2  1  0]
[ 4  3  3  2  1  1  0  1]
[ 1  1  1  1  1  0  1 -2]
  
```

Now remove the vertex v6 from the graph. The tree will look like:



$$D[0,6] < D[0,4]$$

Consider vertex pendant v7.

Perform the following operations:

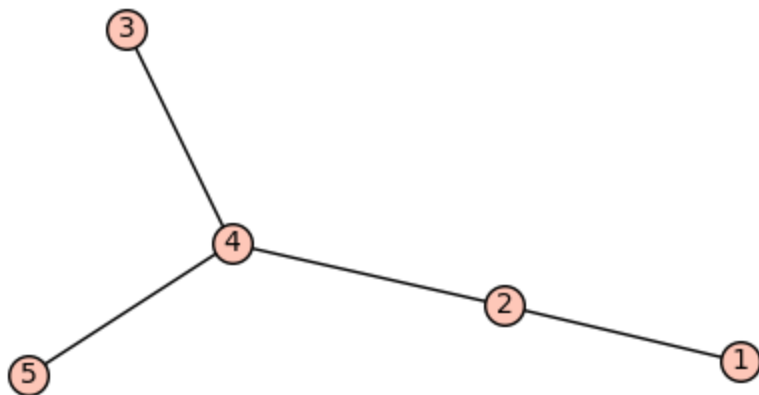
$$C7 \rightarrow C7 - C5$$

$$R7 \rightarrow R7 - R5$$

```

[ 0  1  3  2  3  1  1  1]
[ 1  0  2  1  2  1  1  1]
[ 3  2  0  1  2  1  1  1]
[ 2  1  1  0  1  1  1  1]
[ 3  2  2  1  0  1  1  1]
[ 1  1  1  1  1 -2  0  0]
[ 1  1  1  1  1  0 -2  0]
[ 1  1  1  1  1  0  0 -2]
  
```

Now remove the vertex v7 from the graph. The tree will look like:



$$D[0,4] < D[0,3]$$

Consider vertex pendant v5.

Perform the following operations:

$$C5 \rightarrow C5 - C4$$

$$R5 \rightarrow R5 - R4$$

$$\begin{bmatrix} 0 & 1 & 3 & 2 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 2 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 2 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 1 & 1 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

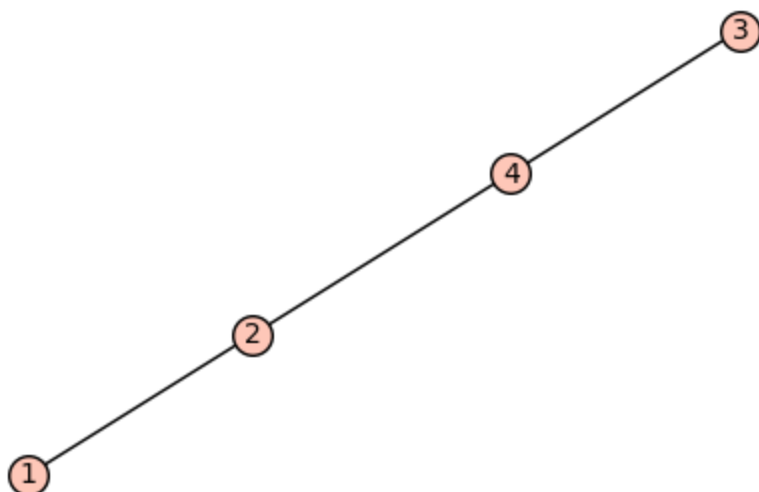
$$\begin{bmatrix} 1 & 1 & 1 & 1 & -2 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & -2 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & -2 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & -2 \end{bmatrix}$$

Now remove the vertex v5 from the graph. The tree will look like:



$$D[0,2] < D[0,3]$$

Consider vertex pendant v3.

Perform the following operations:

$$C3 \rightarrow C3 - C4$$

$$R3 \rightarrow R3 - R4$$

$$\begin{bmatrix} 0 & 1 & 1 & 2 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & -2 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 1 & 1 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

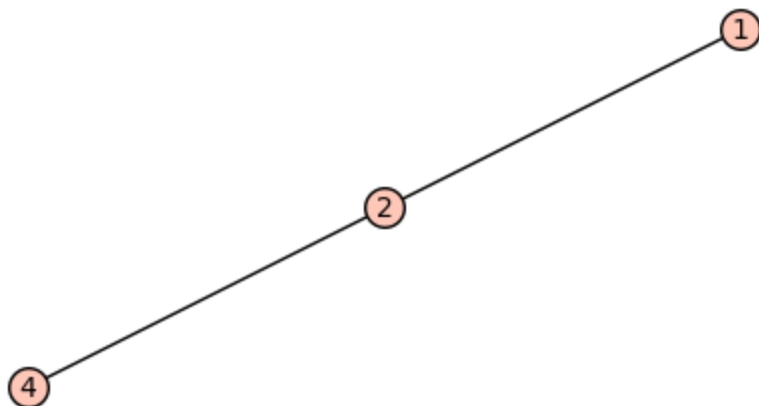
$$\begin{bmatrix} 1 & 1 & 0 & 1 & -2 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 0 & 1 & 0 & -2 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & -2 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 & -2 \end{bmatrix}$$

Now remove the vertex v3 from the graph. The tree will look like:



$$D[0,3] < D[0,1]$$

Consider vertex pendant v4.

Perform the following operations:

$$C4 \rightarrow C4 - C2$$

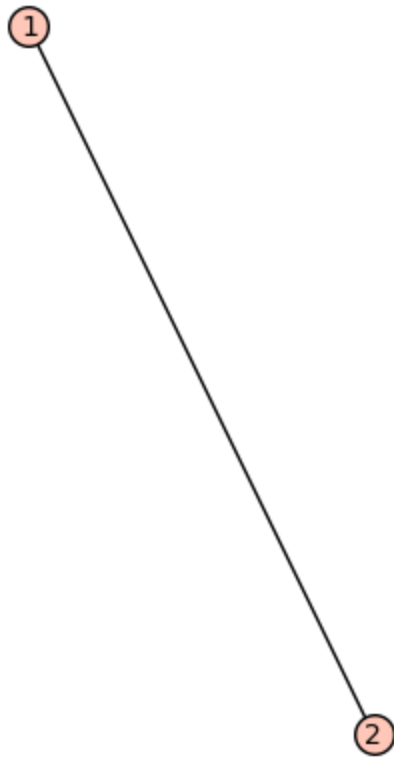
$$R4 \rightarrow R4 - R2$$

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

```
[ 1  1 -2  0  0  0  0  0]
[ 1  1  0 -2  0  0  0  0]
[ 1  1  0  0 -2  0  0  0]
[ 1  1  0  0  0 -2  0  0]
[ 1  1  0  0  0  0 -2  0]
[ 1  1  0  0  0  0  0 -2]
```

Now remove the vertex v4 from the graph. The tree will look like:



$D[0,1] < D[0,0]$

Consider vertex pendant v2.

Perform the following operations:

$C2 \rightarrow C2 - C1$

$R2 \rightarrow R2 - R1$

```
[ 0  1  1  1  1  1  1  1]
[ 1 -2  0  0  0  0  0  0]
[ 1  0 -2  0  0  0  0  0]
[ 1  0  0 -2  0  0  0  0]
[ 1  0  0  0 -2  0  0  0]
[ 1  0  0  0  0 -2  0  0]
[ 1  0  0  0  0  0 -2  0]
[ 1  0  0  0  0  0  0 -2]
```

Now remove the vertex v2 from the graph. The tree will look like:



No vertices with degree 1 found.

let A be:

```
[-2  0  0  0  0  0  0  0]
[ 0 -2  0  0  0  0  0  0]
[ 0  0 -2  0  0  0  0  0]
[ 0  0  0 -2  0  0  0  0]
[ 0  0  0  0 -2  0  0  0]
[ 0  0  0  0  0 -2  0  0]
[ 0  0  0  0  0  0 -2  0]
[ 0  0  0  0  0  0  0 -2]
```

```

let C be:
[1]
[1]
[1]
[1]
[1]
[1]
[1]
[1]
let B be:
[1 1 1 1 1 1 1]
let D be:
0
schur_comp = D - B*A.inverse()* C
D2= det(A) * det(schur_comp)
Finally we have the Determinant of D2 (by Schur complement) : -448.0
The determinant from the formula: -448

```

Let us define the process as a function

```

In [88]: def graham_pollak(n,edges):
          T = Graph(edges)
          print("The graph is given as:")
          T.show()
          D = T.distance_matrix()
          print("The Distance matrix is:")
          print(D)
          for v in T.vertices():
              vertices_with_degree_1 = [v for v in T.vertices() if T.degree(v) == 1]
              if vertices_with_degree_1: # Check if the list is not empty
                  vertex_to_delete = vertices_with_degree_1[-1]
                  print(f" D[0,{vertex_to_delete-1}] < D[0,
{T.neighbors(vertex_to_delete)[0]-1}]")
                  if D[0, vertex_to_delete-1] > D[0, T.neighbors(vertex_to_delete)
[0]-1]:
                      print(f"Consider vertex pendant v{vertex_to_delete}. \nPerform
the following operations:")
                      print(f"C{vertex_to_delete} -> C{vertex_to_delete} -
C{T.neighbors(vertex_to_delete)[0]}")
                      print(f"R{vertex_to_delete}->R{vertex_to_delete} -
R{T.neighbors(vertex_to_delete)[0]}")
                      D[vertex_to_delete-1, :] = D.row(vertex_to_delete-1) -
D.row(T.neighbors(vertex_to_delete)[0]-1)
                      D[:, vertex_to_delete-1] = D.column(vertex_to_delete-1) -
D.column(T.neighbors(vertex_to_delete)[0]-1)
                      print(D)
                      print(f"Now remove the vertex v{vertex_to_delete} from the
graph. The tree will look like:")
                      T.delete_vertex(vertex_to_delete)
                      T.show()

              else:
                  print("No vertices with degree 1 found.") # Handle case with no
degree 1 vertices

          print(f"Finally we have the Determinant of D2 (by Schur complement) :
{schur_determinant(D)}")
          c = (-1)**(n-1) * (n-1) * 2**(n-2)
          print("The determinant from the formula: ", c)

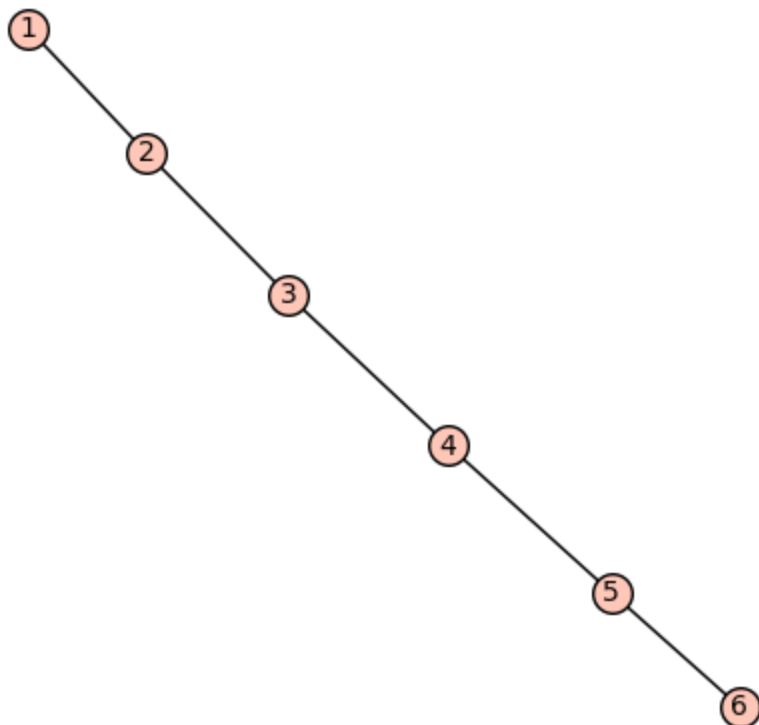
```

```

In [89]: n=6
          edges = [(1,2), (2,3), (3,4), (4,5), (5,6)]
          graham_pollak(n,edges)

```

Out[89]: The graph is given as:



The Distance matrix is:

```

[0 1 2 3 4 5]
[1 0 1 2 3 4]
[2 1 0 1 2 3]
[3 2 1 0 1 2]
[4 3 2 1 0 1]
[5 4 3 2 1 0]

```

$D[0,5] < D[0,4]$

Consider vertex pendant v6.

Perform the following operations:

$C_6 \rightarrow C_6 - C_5$

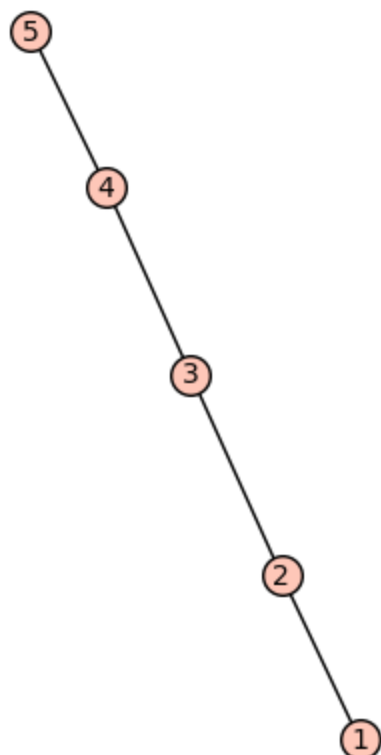
$R_6 \rightarrow R_6 - R_5$

```

[ 0  1  2  3  4  1]
[ 1  0  1  2  3  1]
[ 2  1  0  1  2  1]
[ 3  2  1  0  1  1]
[ 4  3  2  1  0  1]
[ 1  1  1  1  1 -2]

```

Now remove the vertex v6 from the graph. The tree will look like:



$D[0,4] < D[0,3]$

Consider vertex pendant v5.

Perform the following operations:

$C_5 \rightarrow C_5 - C_4$

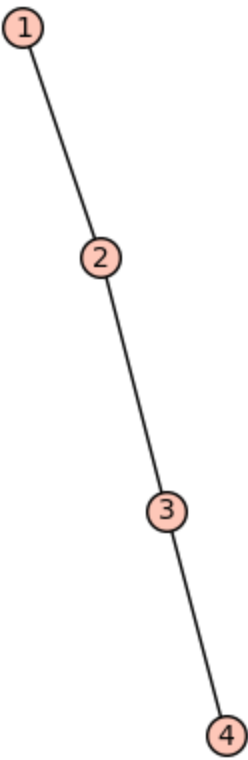
$R_5 \rightarrow R_5 - R_4$

```

[ 0  1  2  3  1  1]
[ 1  0  1  2  1  1]
[ 2  1  0  1  1  1]
[ 3  2  1  0  1  1]
[ 1  1  1  1 -2  0]
[ 1  1  1  1  0 -2]

```

Now remove the vertex v5 from the graph. The tree will look like:



$$D[0,3] < D[0,2]$$

Consider vertex pendant v4.

Perform the following operations:

$$C4 \rightarrow C4 - C3$$

$$R4 \rightarrow R4 - R3$$

$$\begin{bmatrix} 0 & 1 & 2 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

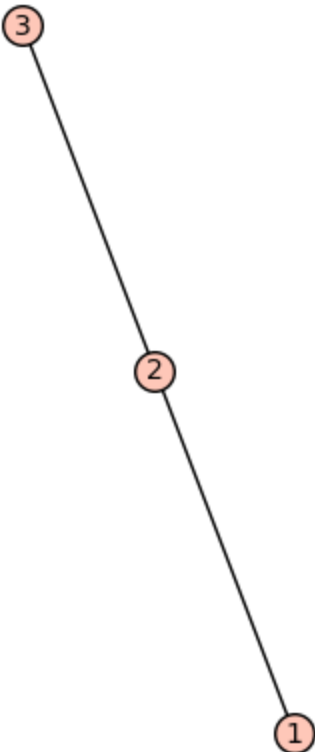
$$\begin{bmatrix} 2 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 & -2 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 & 0 & -2 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 & -2 \end{bmatrix}$$

Now remove the vertex v4 from the graph. The tree will look like:



$$D[0,2] < D[0,1]$$

Consider vertex pendant v3.

Perform the following operations:

$$C3 \rightarrow C3 - C2$$

$$R3 \rightarrow R3 - R2$$

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & -2 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 0 & -2 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 0 & 0 & -2 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 0 & 0 & 0 & -2 \end{bmatrix}$$

Now remove the vertex v3 from the graph. The tree will look like:



$$D[0,1] < D[0,0]$$

Consider vertex pendant v2.

Perform the following operations:

$$C2 \rightarrow C2 - C1$$

$$R2 \rightarrow R2 - R1$$

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & -2 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & -2 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & -2 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & -2 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & -2 \end{bmatrix}$$

Now remove the vertex v2 from the graph. The tree will look like:



No vertices with degree 1 found.

let A be:

$$\begin{bmatrix} -2 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & -2 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & -2 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & -2 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & -2 \end{bmatrix}$$

let C be:

$$\begin{bmatrix} 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 \end{bmatrix}$$

let B be:

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

let D be:

$$0$$

$$\text{schur_comp} = D - B \cdot A.\text{inverse()} \cdot C$$

D2= det(A) * det(schur_comp)
 Finally we have the Determinant of D2 (by Schur complement) : -80.0
 The determinant from the formula: -80

Let us work on a Random tree ie using only the number of vertices
 Note that for random graphs indexing starts from 0 so we make lil changes accordingly.

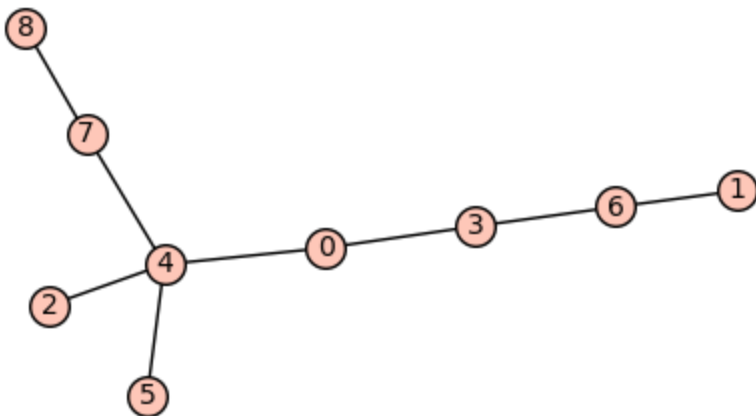
```
In [90]: def graham_pollak_ver(n):
    T =graphs.RandomTree(n)
    print("The graph is given as:")
    T.show()
    D = T.distance_matrix()
    print("The Distance matrix is:")
    print(D)
    for v in T.vertices():
        vertices_with_degree_1 = [v for v in T.vertices() if T.degree(v) == 1]
        if vertices_with_degree_1: # Check if the list is not empty
            vertex_to_delete = vertices_with_degree_1[-1]
            print(f" D[0,{vertex_to_delete}] < D[0,
{T.neighbors(vertex_to_delete)[0]})"]
            if D[0, vertex_to_delete] > D[0, T.neighbors(vertex_to_delete)
[0]]:
                print(f"Consider vertex pendant v{vertex_to_delete}. \nPerform
the following operations:")
                print(f"C{vertex_to_delete} -> C{vertex_to_delete} -
C{T.neighbors(vertex_to_delete)[0]}")
                print(f"R{vertex_to_delete} -> R{vertex_to_delete} -
R{T.neighbors(vertex_to_delete)[0]}")
                D[vertex_to_delete, :] = D.row(vertex_to_delete) -
D.row(T.neighbors(vertex_to_delete)[0])
                D[:, vertex_to_delete] = D.column(vertex_to_delete) -
D.column(T.neighbors(vertex_to_delete)[0])
                print(D)
                print(f"Now remove the vertex v{vertex_to_delete} from the
graph. The tree will look like:")
                T.delete_vertex(vertex_to_delete)
                T.show()

            else:
                print("No vertices with degree 1 found.") # Handle case with no
degree 1 vertices

        print(f"Finally we have the Determinant of D2 (by Schur complement) :
{schur_determinant(D)}")
        c = (-1)**(n-1) * (n-1) * 2**(n-2)
        print("The determinant from the formula: ", c)
```

```
In [92]: graham_pollak_ver(9)
```

Out[92]: The graph is given as:



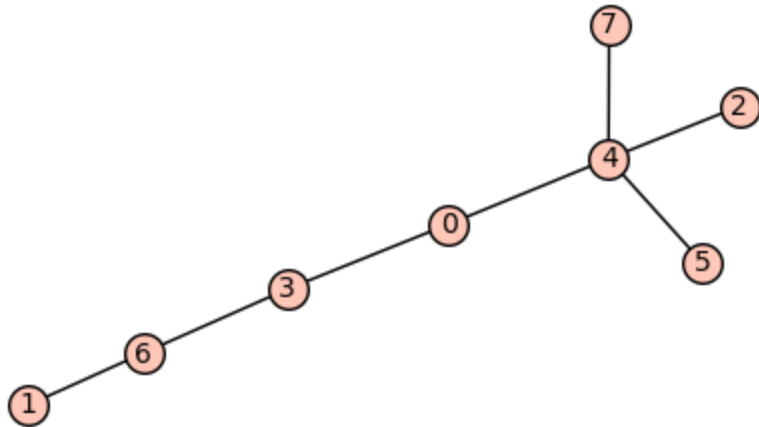
The Distance matrix is:

```
[0 3 2 1 1 2 2 2 3]
[3 0 5 2 4 5 1 5 6]
[2 5 0 3 1 2 4 2 3]
[1 2 3 0 2 3 1 3 4]
[1 4 1 2 0 1 3 1 2]
[2 5 2 3 1 0 4 2 3]
[2 1 4 1 3 4 0 4 5]
[2 5 2 3 1 2 4 0 1]
[3 6 3 4 2 3 5 1 0]
```

$D[0,8] < D[0,7]$
 Consider vertex pendant v8.
 Perform the following operations:
 $C8 \rightarrow C8 - C7$
 $R8 \rightarrow R8 - R7$

[	0	3	2	1	1	2	2	2	1]
[	3	0	5	2	4	5	1	5	1]
[	2	5	0	3	1	2	4	2	1]
[	1	2	3	0	2	3	1	3	1]
[	1	4	1	2	0	1	3	1	1]
[	2	5	2	3	1	0	4	2	1]
[	2	1	4	1	3	4	0	4	1]
[	2	5	2	3	1	2	4	0	1]
[	1	1	1	1	1	1	1	1	-2]

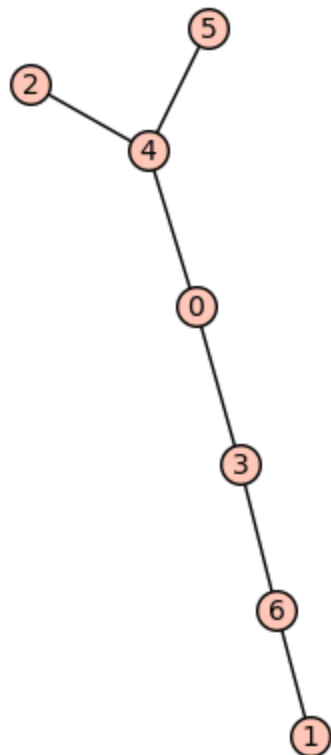
Now remove the vertex v8 from the graph. The tree will look like:



$D[0,7] < D[0,4]$
 Consider vertex pendant v7.
 Perform the following operations:
 $C7 \rightarrow C7 - C4$
 $R7 \rightarrow R7 - R4$

[	0	3	2	1	1	2	2	1	1]
[	3	0	5	2	4	5	1	1	1]
[	2	5	0	3	1	2	4	1	1]
[	1	2	3	0	2	3	1	1	1]
[	1	4	1	2	0	1	3	1	1]
[	2	5	2	3	1	0	4	1	1]
[	2	1	4	1	3	4	0	1	1]
[	1	1	1	1	1	1	1	-2	0]
[	1	1	1	1	1	1	1	0	-2]

Now remove the vertex v7 from the graph. The tree will look like:

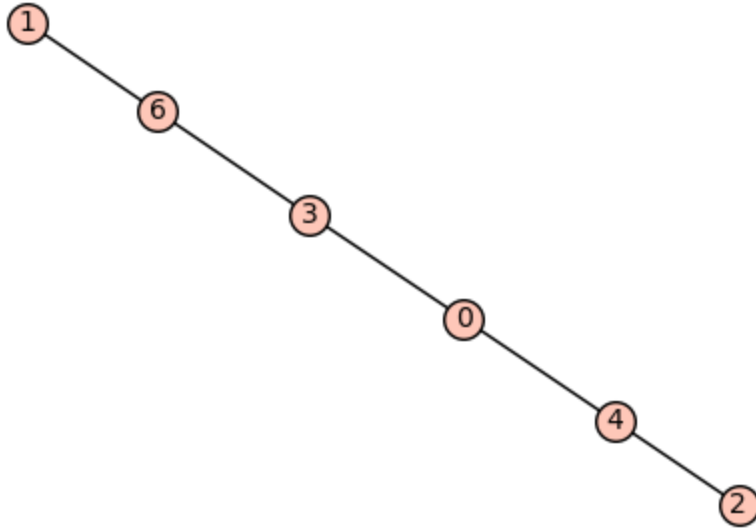


$D[0,5] < D[0,4]$
 Consider vertex pendant v5.
 Perform the following operations:
 $C5 \rightarrow C5 - C4$
 $R5 \rightarrow R5 - R4$

[	0	3	2	1	1	1	2	1	1]
[	3	0	5	2	4	1	1	1	1]
[	2	5	0	3	1	1	4	1	1]
[	1	2	3	0	2	1	1	1	1]

```
[ 1  4  1  2  0  1  3  1  1]
[ 1  1  1  1  1 -2  1  0  0]
[ 2  1  4  1  3  1  0  1  1]
[ 1  1  1  1  1  0  1 -2  0]
[ 1  1  1  1  1  0  1  0 -2]
```

Now remove the vertex v5 from the graph. The tree will look like:



$D[0,2] < D[0,4]$

Consider vertex pendant v2.

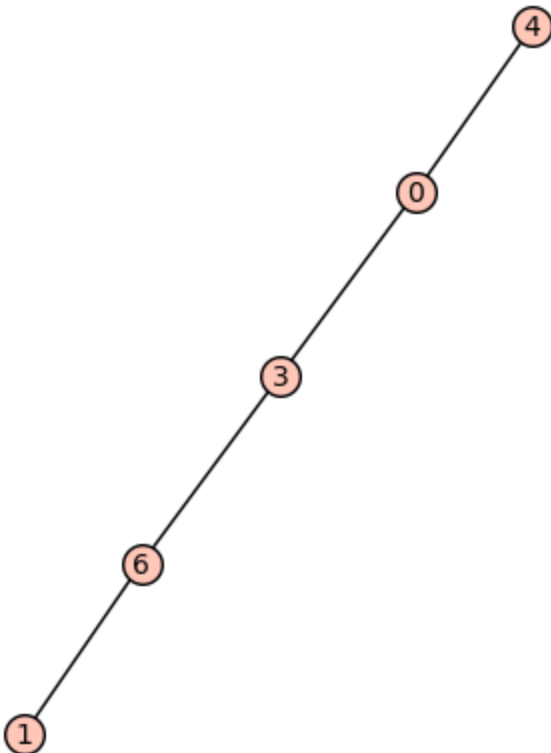
Perform the following operations:

$C2 \rightarrow C2 - C4$

$R2 \rightarrow R2 - R4$

```
[ 0  3  1  1  1  1  2  1  1]
[ 3  0  1  2  4  1  1  1  1]
[ 1  1 -2  1  1  0  1  0  0]
[ 1  2  1  0  2  1  1  1  1]
[ 1  4  1  2  0  1  3  1  1]
[ 1  1  0  1  1 -2  1  0  0]
[ 2  1  1  1  3  1  0  1  1]
[ 1  1  0  1  1  0  1 -2  0]
[ 1  1  0  1  1  0  1  0 -2]
```

Now remove the vertex v2 from the graph. The tree will look like:



$D[0,4] < D[0,0]$

Consider vertex pendant v4.

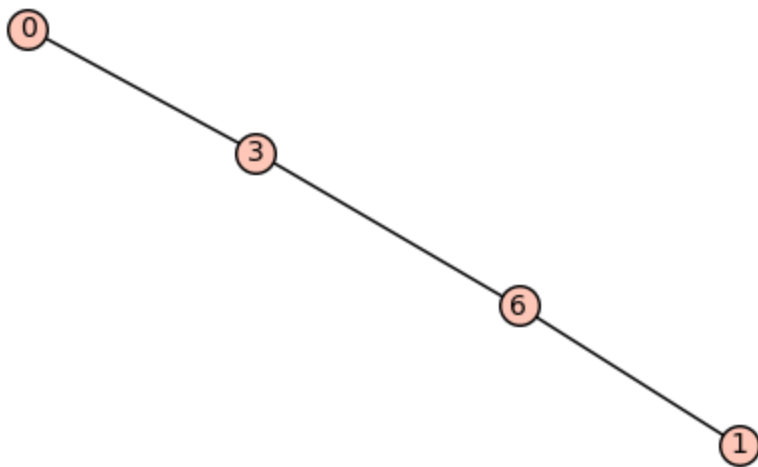
Perform the following operations:

$C4 \rightarrow C4 - C0$

$R4 \rightarrow R4 - R0$

```
[ 0  3  1  1  1  1  2  1  1]
[ 3  0  1  2  1  1  1  1  1]
[ 1  1 -2  1  0  0  1  0  0]
[ 1  2  1  0  1  1  1  1  1]
[ 1  1  0  1 -2  0  1  0  0]
[ 1  1  0  1  0 -2  1  0  0]
[ 2  1  1  1  1  1  0  1  1]
[ 1  1  0  1  0  0  1 -2  0]
[ 1  1  0  1  0  0  1  0 -2]
```

Now remove the vertex v4 from the graph. The tree will look like:



$$D[0,1] < D[0,6]$$

Consider vertex pendant v1.

Perform the following operations:

$$C1 \rightarrow C1 - C6$$

$$R1 \rightarrow R1 - R6$$

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 & 2 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & -2 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & -2 & 1 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 1 & -2 & 0 & 1 & 0 & 0 \end{bmatrix}$$

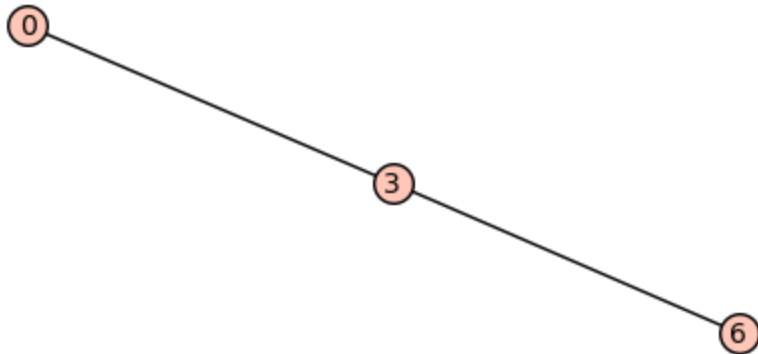
$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & -2 & 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 & -2 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & -2 \end{bmatrix}$$

Now remove the vertex v1 from the graph. The tree will look like:



$$D[0,6] < D[0,3]$$

Consider vertex pendant v6.

Perform the following operations:

$$C6 \rightarrow C6 - C3$$

$$R6 \rightarrow R6 - R3$$

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & -2 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & -2 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 1 & -2 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & -2 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & -2 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 0 & -2 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & -2 \end{bmatrix}$$

Now remove the vertex v6 from the graph. The tree will look like:



$$D[0,3] < D[0,0]$$

Consider vertex pendant v3.

Perform the following operations:

$$C3 \rightarrow C3 - C0$$

$$R3 \rightarrow R3 - R0$$

```

[ 0  1  1  1  1  1  1  1  1]
[ 1 -2  0  0  0  0  0  0  0]
[ 1  0 -2  0  0  0  0  0  0]
[ 1  0  0 -2  0  0  0  0  0]
[ 1  0  0  0 -2  0  0  0  0]
[ 1  0  0  0  0 -2  0  0  0]
[ 1  0  0  0  0  0 -2  0  0]
[ 1  0  0  0  0  0  0 -2  0]
[ 1  0  0  0  0  0  0  0 -2]
  
```

Now remove the vertex v3 from the graph. The tree will look like:



No vertices with degree 1 found.

let A be:

```

[-2  0  0  0  0  0  0  0]
[ 0 -2  0  0  0  0  0  0]
[ 0  0 -2  0  0  0  0  0]
[ 0  0  0 -2  0  0  0  0]
[ 0  0  0  0 -2  0  0  0]
[ 0  0  0  0  0 -2  0  0]
[ 0  0  0  0  0  0 -2  0]
[ 0  0  0  0  0  0  0 -2]
  
```

let C be:

```

[1]
[1]
[1]
[1]
  
```

```

[1]
[1]
[1]
[1]
let B be:
[1 1 1 1 1 1 1 1]
let D be:
0
schur_comp = D - B*A.inverse()* C
D2= det(A) * det(schur_comp)
Finally we have the Determinant of D2 (by Schur complement) : 1024.0
The determinant from the formula: 1024

```

Why does this happens? (Intuition)

A tree might look very different depending on how its edges connect, but all trees of the same size share a few universal properties:

- There are exactly $n-1$ edges.
- There's a unique simple path between any two vertices (because no cycles).
- The way distances "flow" across the tree has a very systematic pattern.

Now, if you cleverly express the distance matrix D in terms of simpler matrices that depend on edges, you find that those dependencies cancel out in such a way that only n , the number of nodes, matters — not which tree you used. The tree distance matrix is a sum of simple rank-1 contributions from edges. Different trees with the same n have the same number of edges and the same algebraic relationships among those edge vectors. These relationships ensure that all the distance matrices are equivalent up to transformations that do not change their determinant. That's what Graham and Pollak demonstrated elegantly — the matrix operations "average out" all the specific geometry of the tree, leaving only the universal number.